



DSA0602-DATA HANDLING AND DATA VISUALIZATION

SLOT A

Dr. ALMAS BEGUM

PRESENTED BY:

SURU SAI KEERTHI [192424368]

Class Activity: 15.07.2026

1.Smart City Air Quality Monitoring: A city has installed 150 air quality sensors across different locations. The sensors record PM2.5, NO2, CO, temperature, and humidity every hour. Decision-makers struggle to identify pollution hotspots because of overlapping values.

CODE:

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt import seaborn as sns data = { "Location": ["L1", "L2", "L3", "L4", "L5", "L6", "L7", "L8", "L9", "L10"], "PM2.5": [35,45,60,85,95,110,125,140,70,55], "NO2": [15,20,22,28,35,40,45,50,30,25], "CO": [0.5,0.6,0.8,1.0,1.2,1.5,1.8,2.0,1.1,0.9], "Temperature": [28,30,32,35,31,33,36,37,29,30], "Humidity": [70,68,65,60,58,55,52,50,66,69] } df = pd.DataFrame(data) plt.figure(figsize=(18,5)) # ----- # 1. Sequential Color Palette # ----- plt.subplot(1,3,1) scatter = plt.scatter(df["Temperature"], df["PM2.5"], c=df["PM2.5"], cmap="Reds", s=180, edgecolors="black"</pre>	<pre>install.packages("ggplot2") library(ggplot2) air <- data.frame(Location = paste("L", 1:15, sep=""), PM25 = c(35,45,60,80,95,110,125,140,75,55,130,150,90,65,100), NO2 = c(15,18,22,28,32,38,45,50,30,25,48,55,35,20,40), CO = c(0.5,0.6,0.8,1.0,1.2,1.5,1.8,2.0,1.1,0.9,2.2,2.5,1.4,0.7,1 .6), Temperature = c(28,29,31,32,33,34,35,36,30,29,37,38,33,31,34), Humidity = c(70,68,65,63,60,58,55,52,66,69,50,48,59,64,57)) ##### ##### # 1. Sequential Color Palette ##### ##### ggplot(air, aes(x=Temperature, y=PM25, color=PM25))+ geom_point(size=5)+ geom_text(aes(label=Location), vjust=-1, size=3)+ scale_color_gradient(low="lightyellow", high="red")+ labs(title="Sequential Color Scale", x="Temperature (°C)",</pre>

```

)

for i in range(len(df)):
    plt.text(
        df["Temperature"][i]+0.1,
        df["PM2.5"][i],
        df["Location"][i],
        fontsize=8
    )

plt.colorbar(scatter,label="PM2.5")
plt.title("Sequential Palette")
plt.xlabel("Temperature (°C)")
plt.ylabel("PM2.5")
plt.grid(True,alpha=0.3)

# -----
# 2. Diverging Color Palette
# -----

plt.subplot(1,3,2)

scatter = plt.scatter(
    df["Temperature"],
    df["PM2.5"],
    c=df["PM2.5"]-100,
    cmap="coolwarm",
    s=180,
    edgecolors="black"
)

for i in range(len(df)):
    plt.text(
        df["Temperature"][i]+0.1,
        df["PM2.5"][i],
        df["Location"][i],
        fontsize=8
    )

plt.colorbar(scatter,label="PM2.5")
plt.title("Diverging Palette")
plt.xlabel("Temperature (°C)")
plt.ylabel("PM2.5")
plt.grid(True,alpha=0.3)

# -----
# 3. Qualitative Color Palette
# -----

plt.subplot(1,3,3)

conditions = []

for value in df["PM2.5"]:
    if value < 50:
        conditions.append("Low")
    elif value < 100:

```

```

    y="PM2.5"
)+
theme_minimal()
#####
#####
# 2. Diverging Color Palette
#####
#####
ggplot(air, aes(x=Temperature, y=PM25, color=PM25-
100))+
    geom_point(size=5)+
    geom_text(aes(label=Location), vjust=-1, size=3)+
    scale_color_gradient2(
        low="blue",
        mid="white",
        high="red",
        midpoint=0
    )+
labs(
    title="Diverging Color Scale",
    x="Temperature (°C)",
    y="PM2.5"
)+
theme_minimal()
#####
#####
# 3. Qualitative Color Palette
#####
#####
air$Pollution <- ifelse(
    air$PM25 < 50,
    "Low",
    ifelse(
        air$PM25 < 100,
        "Moderate",
        "High"
    )
)

ggplot(air, aes(x=Temperature, y=PM25,
color=Pollution))+
    geom_point(size=5)+
    geom_text(aes(label=Location), vjust=-1, size=3)+
    labs(
        title="Qualitative Color Scale",
        x="Temperature (°C)",
        y="PM2.5"
    )+
theme_minimal()

```

```
        conditions.append("Moderate")
    else:
        conditions.append("High")

df["Pollution"] = conditions

colors = {
    "Low": "green",
    "Moderate": "orange",
    "High": "red"
}

for category in colors:
    temp = df[df["Pollution"] == category]

    plt.scatter(
        temp["Temperature"],
        temp["PM2.5"],
        s=180,
        color=colors[category],
        edgecolors="black",
        label=category
    )

for i in range(len(df)):
    plt.text(
        df["Temperature"][i]+0.1,
        df["PM2.5"][i],
        df["Location"][i],
        fontsize=8
    )

plt.title("Qualitative Palette")
plt.xlabel("Temperature (°C)")
plt.ylabel("PM2.5")
plt.legend(title="Pollution")
plt.grid(True,alpha=0.3)

plt.tight_layout()
plt.show()
```

OUTPUT:



Comparison

Palette	Best Use	Interpretation
Sequential	Continuous values	Shows gradual increase in PM2.5 pollution. Darker red indicates higher pollution.
Diverging	Comparing against a threshold	Blue shows values below the threshold (100), red shows values above it.
Qualitative	Categories	Classifies pollution into Low , Moderate , and High groups.

Justification

The Sequential Colour Palette is the most effective for identifying pollution hotspots because PM2.5 is a continuous measurement. The gradual colour transition makes it easy to spot areas with increasing pollution intensity. The diverging palette is useful when comparing values against a standard threshold, while the qualitative palette is best for simple categorical communication.

2. Earthquake Risk Analysis: A geological survey contains earthquake magnitude (0.5–9.5), depth (1–700 km), latitude, longitude, and aftershock frequency. Small earthquakes dominate the dataset, making large events difficult to interpret.

CODE:

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt data = { "Magnitude": [0.8, 1.2, 1.8, 2.5, 3.0, 3.8, 4.5, 5.2, 6.0, 6.8, 7.5, 8.3, 9.0], "Depth": [5, 12, 20, 35, 50, 80, 120, 180, 250, 350, 450, 550, 700], "Latitude": [10, 12, 15, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36], "Longitude": [70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94], "Aftershock": [2, 4, 8, 12, 18, 30, 55, 90, 150, 250, 400, 650, 1000] } df = pd.DataFrame(data) plt.figure(figsize=(14, 6)) ===== # 1. Linear Scale # ===== plt.subplot(1, 2, 1)</pre>	<pre>install.packages("ggplot2") library(ggplot2) earthquake <- data.frame(Magnitude = c(0.8, 1.2, 1.8, 2.5, 3.0, 3.8, 4.5, 5.2, 6.0, 6.8, 7.5, 8.3, 9.0), Depth = c(5, 12, 20, 35, 50, 80, 120, 180, 250, 350, 450, 550, 700), Latitude = c(10, 12, 15, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36), Longitude = c(70, 72, 74, 76, 78, 80, 82, 84, 86, 88, 90, 92, 94), Aftershock = c(2, 4, 8, 12, 18, 30, 55, 90, 150, 250, 400, 650, 1000)) ##### ##### # 1. Linear Scale ##### ##### ggplot(earthquake, aes(x = Magnitude, y = Aftershock)) + geom_point(color="blue", size=4) + geom_line(color="darkblue", linewidth=1) + geom_text(aes(label=Magnitude), vjust=-1, size=3) + labs(</pre>

```

plt.plot(
    df["Magnitude"],
    df["Aftershock"],
    marker="o",
    linewidth=2,
    color="royalblue"
)

for i in range(len(df)):
    plt.text(
        df["Magnitude"][i],
        df["Aftershock"][i]+20,
        str(df["Magnitude"][i]),
        fontsize=8
    )

plt.title("Earthquake Distribution (Linear Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency")
plt.grid(True,alpha=0.3)

#
=====
=====
# 2. Logarithmic Scale
#
=====
=====

plt.subplot(1,2,2)

plt.plot(
    df["Magnitude"],
    df["Aftershock"],
    marker="o",
    linewidth=2,
    color="crimson"
)

for i in range(len(df)):
    plt.text(
        df["Magnitude"][i],
        df["Aftershock"][i],
        str(df["Magnitude"][i]),
        fontsize=8
    )

plt.yscale("log")

plt.title("Earthquake Distribution (Logarithmic
Scale)")
plt.xlabel("Magnitude")
plt.ylabel("Aftershock Frequency (Log Scale)")
plt.grid(True,which="both",alpha=0.3)

```

```

title="Earthquake Distribution (Linear Scale)",
x="Earthquake Magnitude",
y="Aftershock Frequency"
) +
theme_minimal()

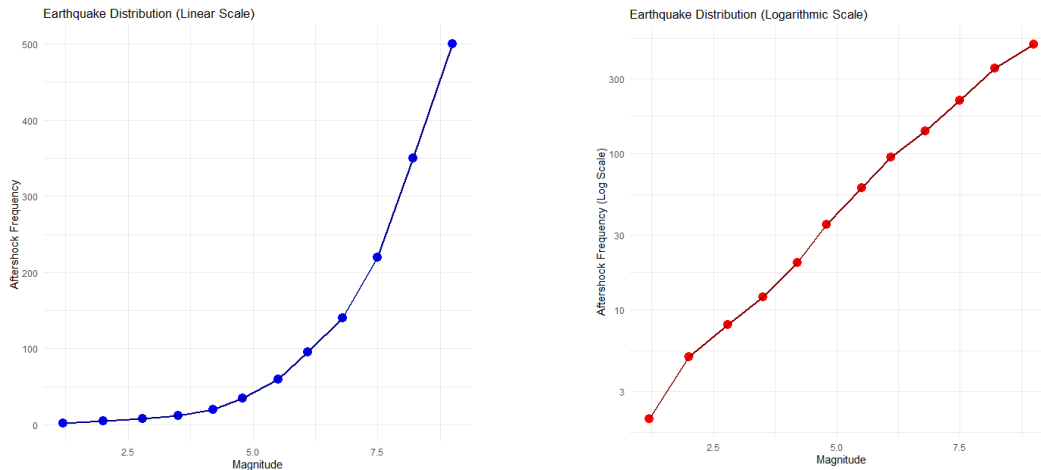
#####
#####
# 2. Logarithmic Scale
#####
#####

ggplot(earthquake,
    aes(x = Magnitude,
        y = Aftershock)) +
geom_point(color="red", size=4) +
geom_line(color="darkred", linewidth=1) +
geom_text(aes(label=Magnitude),
    vjust=-1,
    size=3) +
scale_y_log10() +
labs(
    title="Earthquake Distribution (Logarithmic
Scale)",
    x="Earthquake Magnitude",
    y="Aftershock Frequency (Log Scale)"
) +
theme_minimal()

```

```
plt.tight_layout()
plt.show()
```

OUTPUT:



Interpretation

Linear Scale

- Small earthquakes are crowded near the bottom.
- Large earthquakes dominate the graph.
- Difficult to compare smaller events.

Logarithmic (Log) Scale

- Spreads out small earthquake values.
- Makes both small and large earthquakes visible.
- Easier to observe trends across the full range of magnitudes.

Conclusion

The **logarithmic scale** provides a better visualization because it reduces the dominance of very large aftershock frequencies and makes smaller earthquake events easier to compare. It is especially useful for datasets that span several orders of magnitude.

3. Wind Direction Analysis for Airport Operations: An airport collects hourly wind speed and wind direction for one year. Pilots need an intuitive visualization showing prevailing wind directions and seasonal variations.

CODE:

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt import numpy as np data = { "Direction": [0,30,60,90,120,150,180,210,240,270, 300,330],</pre>	<pre>library(ggplot2) wind <- data.frame(Direction = c(0,30,60,90,120,150,180,210,240,270,300,330), WindSpeed = c(18,22,28,35,30,26,20,16,14,19,24,29))</pre>

```
"WindSpeed": [18,22,28,35,30,26,20,16,14,19,24,29]
}
```

```
df = pd.DataFrame(data)
```

```
plt.figure(figsize=(15,6))
```

```
=====
# 1. Cartesian Plot
```

```
plt.subplot(1,2,1)
```

```
plt.plot(
    df["Direction"],
    df["WindSpeed"],
    marker='o',
    linewidth=2,
    color="royalblue"
)
```

```
for i in range(len(df)):
    plt.text(
        df["Direction"][i],
        df["WindSpeed"][i]+0.7,
        str(df["Direction"][i])+"°",
        fontsize=8
    )
```

```
plt.title("Wind Direction Analysis (Cartesian Plot)")
plt.xlabel("Wind Direction (Degrees)")
plt.ylabel("Wind Speed (km/h)")
plt.grid(alpha=0.3)
```

```
#
```

```
=====
# 2. Polar Coordinate (Wind Rose)
```

```
#
```

```
ax = plt.subplot(1,2,2, projection='polar')
```

```
theta = np.deg2rad(df["Direction"])
```

```
bars = ax.bar(
    theta,
    df["WindSpeed"],
    width=np.deg2rad(25),
```

```
color=plt.cm.viridis(df["WindSpeed"]/max(df["WindSpeed"])),
    edgecolor="black"
```

```
#####
#####
# 1. Cartesian Plot
#####
#####
```

```
ggplot(wind, aes(x = Direction, y = WindSpeed)) +
    geom_line(color = "blue", linewidth = 1.2) +
    geom_point(color = "red", size = 4) +
    geom_text(aes(label = Direction),
              vjust = -1,
              size = 3.5) +
```

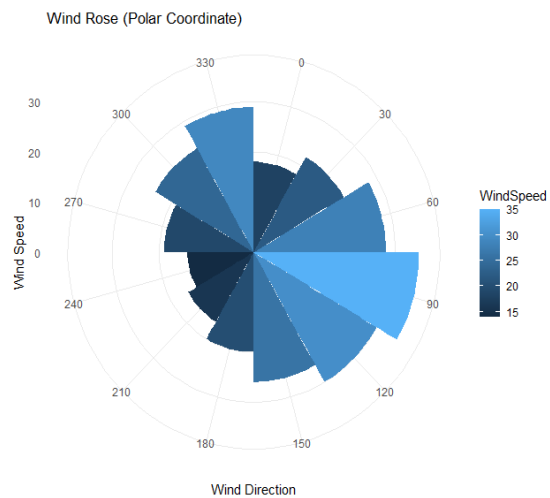
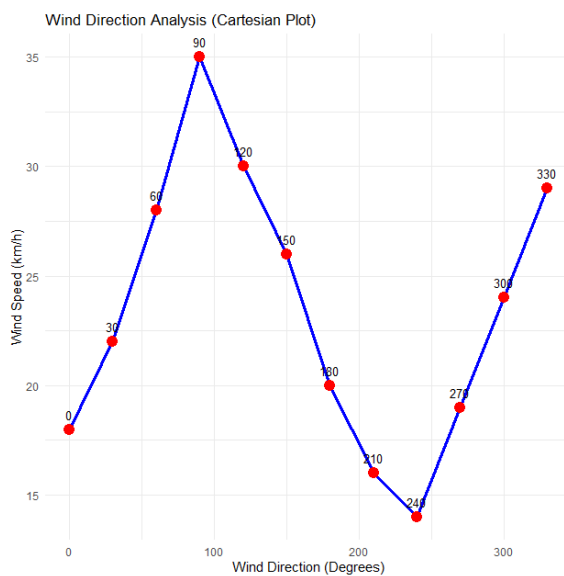
```
labs(
    title = "Wind Direction Analysis (Cartesian Plot)",
    x = "Wind Direction (Degrees)",
    y = "Wind Speed (km/h)"
) +
theme_minimal()
```

```
#####
#####
# 2. Polar Coordinate (Wind Rose)
#####
#####
```

```
ggplot(wind,
    aes(x = factor(Direction),
        y = WindSpeed,
        fill = WindSpeed)) +
    geom_bar(stat = "identity",
            width = 1) +
    coord_polar() +
    labs(
        title = "Wind Rose (Polar Coordinate)",
        x = "Wind Direction",
        y = "Wind Speed"
    ) +
    theme_minimal()
```

```
)  
  
ax.set_title("Wind Rose (Polar Coordinate)")  
ax.set_theta_zero_location("N")  
ax.set_theta_direction(-1)  
  
plt.tight_layout()  
plt.show()
```

OUTPUT:



Interpretation

Cartesian Plot

- Displays wind speed against wind direction.
- Easy to compare wind speeds numerically.
- Seasonal or directional patterns are less obvious.

Polar Coordinate (Wind Rose)

- Clearly shows prevailing wind directions.

- Longer bars indicate stronger winds.
- Better represents circular directional data.

Evaluation

The **Wind Rose (Polar Coordinate)** is the better visualization for airport operations because wind direction is circular by nature. Pilots and airport authorities can quickly identify the dominant wind directions and make runway alignment decisions more effectively than with a standard Cartesian plot.

4. Hospital Patient Monitoring Dashboard: ICU patients have heart rate, oxygen saturation, blood pressure, respiration rate, and temperature recorded every minute. Doctors require a dashboard that instantly highlights abnormal conditions.

CODE:

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt # Create Dataset data = { "Patient":["P1","P2","P3","P4","P5","P6","P7","P8","P9", "P10"], "HeartRate":[72,95,120,60,110,80,135,70,88,125], "Oxygen":[98,96,91,99,88,97,85,98,95,89], "BloodPressure":[120,125,145,115,150,118,160,122,128, 155], "Respiration":[18,20,28,16,30,19,32,17,21,29], "Temperature":[36.8,37.2,38.8,36.5,39.1,37.0,40.0,36.9,3 7.5,38.5] } df = pd.DataFrame(data) # Create Status Column status = [] for oxygen in df["Oxygen"]: if oxygen >= 95: status.append("Normal") elif oxygen >= 90:</pre>	<pre>library(ggplot2) patient <- data.frame(Patient = c("P1","P2","P3","P4","P5","P6","P7","P8","P9", "P10"), HeartRate = c(72,95,120,60,110,80,135,70,88,125), Oxygen = c(98,96,91,99,88,97,85,98,95,89), BloodPressure = c(120,125,145,115,150,118,160,122,128,155), Respiration = c(18,20,28,16,30,19,32,17,21,29), Temperature = c(36.8,37.2,38.8,36.5,39.1,37.0,40.0,36.9,37.5,38 .5)) # Status based on Oxygen Saturation patient\$Status <- ifelse(patient\$Oxygen >= 95, "Normal", ifelse(patient\$Oxygen >= 90, "Warning", "Critical")) ggplot(patient, aes(x = Patient, y = HeartRate, color = Status, size = Temperature)) +</pre>

```

        status.append("Warning")
    else:
        status.append("Critical")

df["Status"] = status

# Color Dictionary
colors = {
    "Normal": "green",
    "Warning": "orange",
    "Critical": "red"
}

#
=====
=
# Plot Dashboard
#
=====
=

plt.figure(figsize=(12,6))

for i in range(len(df)):
    plt.scatter(
        df["Patient"][i],
        df["HeartRate"][i],
        s=df["Temperature"][i]*18,
        color=colors[df["Status"][i]],
        edgecolor="black"
    )

    plt.text(
        df["Patient"][i],
        df["HeartRate"][i]+2,
        df["Patient"][i],
        fontsize=9,
        ha="center"
    )

plt.title("Hospital Patient Monitoring Dashboard")
plt.xlabel("Patient ID")
plt.ylabel("Heart Rate (BPM)")

# Legend
plt.scatter([],[],color="green",s=120,label="Normal")
plt.scatter([],[],color="orange",s=120,label="Warning")
plt.scatter([],[],color="red",s=120,label="Critical")

plt.legend(title="Patient Status")
plt.grid(alpha=0.3)

plt.show()

```

```

geom_point() +

geom_text(aes(label = Patient),
          vjust = -1,
          size = 4) +

scale_color_manual(values = c(
    "Normal" = "green",
    "Warning" = "orange",
    "Critical" = "red"
)) +

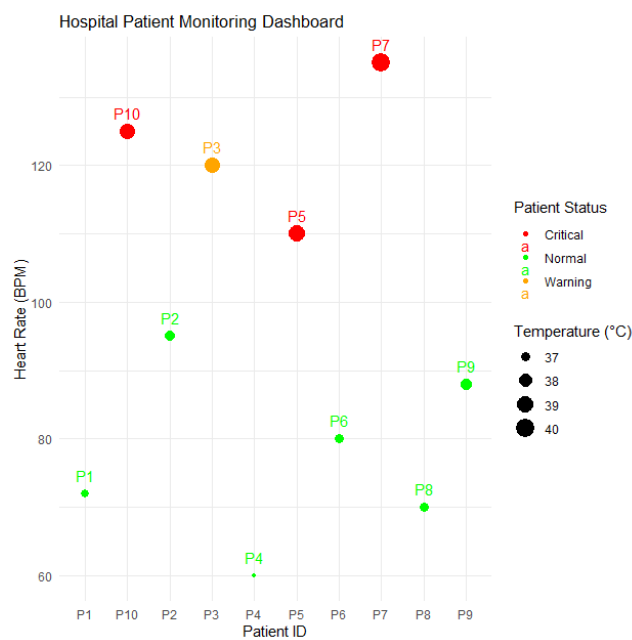
labs(
    title = "Hospital Patient Monitoring
Dashboard",
    x = "Patient ID",
    y = "Heart Rate (BPM)",
    color = "Patient Status",
    size = "Temperature (°C)"
) +

theme_minimal()

```

--	--

OUTPUT:



Thresholds Used

Parameter	Normal	Warning	Critical
Oxygen Saturation	≥95%	90–94%	<90%
Heart Rate	60–100 BPM	101–120 BPM	>120 BPM
Temperature	36.5–37.5°C	37.6–38.5°C	>38.5°C

Why These Colors?

- **Green** → Normal condition; calming and indicates no immediate concern.
- **Orange** → Warning; draws attention without causing alarm.

- **Red** → Critical; immediately signals that urgent medical action may be needed.

Using these colors helps doctors quickly identify patients requiring attention while avoiding visual overload from too many intense colors.

5. Global Climate Change Study: Researchers analyze temperature anomalies across 180 countries from 1980–2025. The objective is to show both positive and negative deviations from historical averages.

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt import seaborn as sns # Create Dataset data = { "Country":["India","USA","China","Brazil","Australia"], "1980":[-0.5,-0.3,-0.4,-0.2,-0.6], "1990":[-0.2,0.1,-0.1,0.0,-0.3], "2000":[0.3,0.5,0.4,0.2,0.1], "2010":[0.8,1.0,0.9,0.6,0.5], "2025":[1.5,1.8,1.6,1.2,1.0] } df = pd.DataFrame(data) # Set Country as Index df.set_index("Country", inplace=True) plt.figure(figsize=(18,5)) # ===== == # 1. Blue - White - Red # ===== == plt.subplot(1,3,1)</pre>	<pre># Install packages (Run only once) install.packages("ggplot2") install.packages("tidyr") # Load libraries library(ggplot2) library(tidyr) # Dataset climate <- data.frame(Country = c("India","USA","China","Brazil","Australia"), Y1980 = c(-0.5,-0.3,-0.4,-0.2,-0.6), Y1990 = c(-0.2,0.1,-0.1,0.0,-0.3), Y2000 = c(0.3,0.5,0.4,0.2,0.1), Y2010 = c(0.8,1.0,0.9,0.6,0.5), Y2025 = c(1.5,1.8,1.6,1.2,1.0)) # Convert Wide to Long Format climate_long <- pivot_longer(climate, cols = starts_with("Y"), names_to = "Year", values_to = "Temperature") # Remove Y from Year climate_long\$Year <- gsub("Y","",climate_long\$Year)</pre>

```
sns.heatmap(
    df,
    cmap="coolwarm",
    annot=True,
    linewidths=0.5,
    center=0,
    cbar_kws={"label": "Temperature Anomaly"}
)
```

```
plt.title("Blue-White-Red Palette")
```

```
#
```

```
=====
```

```
==
```

```
# 2. Purple - White - Green
```

```
#
```

```
=====
```

```
plt.subplot(1,3,2)
```

```
sns.heatmap(
    df,
    cmap="PRGn",
    annot=True,
    linewidths=0.5,
    center=0,
    cbar_kws={"label": "Temperature Anomaly"}
)
```

```
plt.title("Purple-White-Green Palette")
```

```
#
```

```
=====
```

```
==
```

```
# 3. Brown - White - Blue
```

```
#
```

```
=====
```

```
plt.subplot(1,3,3)
```

```
sns.heatmap(
    df,
    cmap="BrBG",
    annot=True,
    linewidths=0.5,
    center=0,
    cbar_kws={"label": "Temperature Anomaly"}
)
```

```
plt.title("Brown-White-Blue Palette")
```

```
plt.tight_layout()
```

```
plt.show()
```

```
#####
##
# Heatmap 1 : Blue - White - Red
#####
##
```

```
ggplot(climate_long,
    aes(x=Year,
        y=Country,
        fill=Temperature))+
    geom_tile(color="white")+
    scale_fill_gradient2(
        low="blue",
        mid="white",
        high="red",
        midpoint=0
    )+
    labs(
        title="Blue - White - Red Heatmap",
        fill="Temperature"
    )+
    theme_minimal()
```

```
#####
##
# Heatmap 2 : Green - White - Purple
#####
##
```

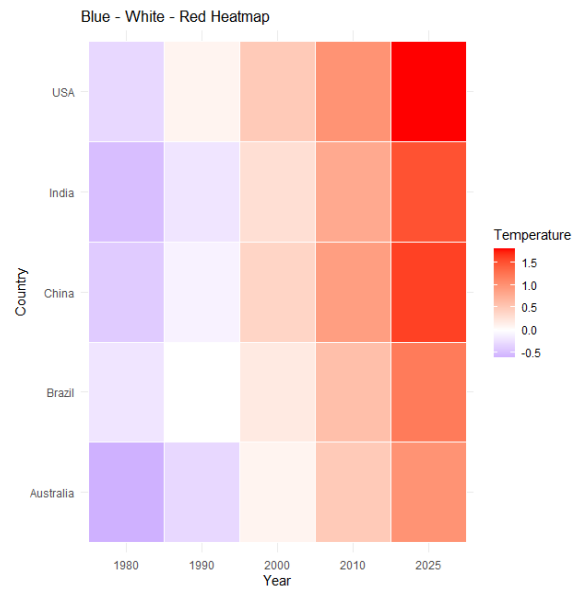
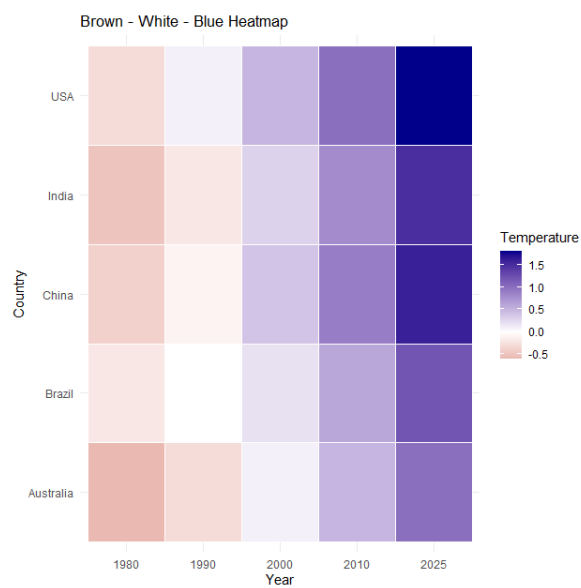
```
ggplot(climate_long,
    aes(x=Year,
        y=Country,
        fill=Temperature))+
    geom_tile(color="white")+
    scale_fill_gradient2(
        low="darkgreen",
        mid="white",
        high="purple",
        midpoint=0
    )+
    labs(
        title="Green - White - Purple Heatmap",
        fill="Temperature"
    )+
    theme_minimal()
```

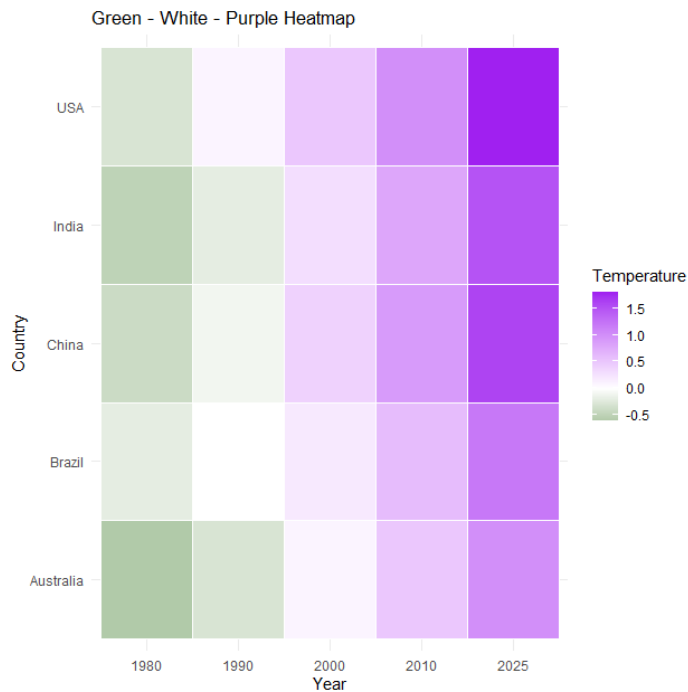
```
#####
##
# Heatmap 3 : Brown - White - Blue
#####
##
```

```
ggplot(climate_long,
    aes(x=Year,
        y=Country,
        fill=Temperature))+
```

```
geom_tile(color="white")+
scale_fill_gradient2(
  low="brown",
  mid="white",
  high="darkblue",
  midpoint=0
)+
labs(
  title="Brown - White - Blue Heatmap",
  fill="Temperature"
)+
theme_minimal()
```

OUTPUT:





Accessibility Considerations

- Use high-contrast colours.
- Keep white as the midpoint (zero anomaly).
- Avoid relying only on red–green combinations because they are difficult for people with red–green colour vision deficiency.
- Include a colour legend and numerical values whenever possible.

Conclusion

Blue–White–Red is the most effective diverging palette because:

- Blue naturally represents cooler-than-average temperatures.
- Red naturally represents warmer-than-average temperatures.
- White clearly indicates the historical average (0 anomaly).
- It provides good contrast and is easier to interpret than the other palettes.

6.E-Commerce Sales Analytics: A retail company sells over 50 product categories across 30 regions. Management wants to distinguish categories while simultaneously representing sales volume.

PYTHON	R PROGRAMMING
import pandas as pd import matplotlib.pyplot as plt import seaborn as sns	# Install packages (Run only once) install.packages("ggplot2")

```
"Category":["Electronics","Clothing","Furniture","Groceries","Sports",
"Books","Toys","Beauty","Footwear","Accessories"],
"Region":["North","South","East","West","North",
"South","East","West","North","South"],
"Sales":[85000,62000,73000,95000,58000,
47000,52000,61000,69000,56000]
}
```

```
df = pd.DataFrame(data)
```

```
plt.figure(figsize=(14,6))
```

```
#
```

```
=====
=====
# 1. Categories using Color
```

```
#
```

```
=====
=====
plt.subplot(1,2,1)
```

```
sns.barplot(
    data=df,
    x="Category",
    y="Sales",
    hue="Category",
    palette="Set2",
    legend=False
)
```

```
plt.title("Product Categories")
plt.xlabel("Category")
plt.ylabel("Sales (₹)")
plt.xticks(rotation=45)
```

```
#
```

```
=====
=====
# 2. Sales Volume using Color Gradient
```

```
#
```

```
=====
=====
plt.subplot(1,2,2)
```

```
bars = plt.bar(
    df["Category"],
    df["Sales"],
    color=plt.cm.Blues(df["Sales"]/df["Sales"].max())
)
```

```
plt.title("Sales Volume by Product Category")
```

```
library(ggplot2)
```

```
# Create Dataset
sales <- data.frame(
```

```
  Category =
c("Electronics","Clothing","Furniture","Groceries","Sports",
```

```
"Books","Toys","Beauty","Footwear","Accessories")
```

```
,
  Region = c("North","South","East","West","North",
"South","East","West","North","South"),
  Sales = c(85000,62000,73000,95000,58000,
47000,52000,61000,69000,56000)
```

```
)
```

```
#####
```

```
#####
```

```
# Visualization 1 : Categories using Color
```

```
#####
```

```
#####
```

```
ggplot(sales,
  aes(x=Category,
      y=Sales,
      fill=Category))+
  geom_bar(stat="identity")+
  labs(
    title="Product Categories",
    x="Category",
    y="Sales (₹)"
  )+
  theme_minimal()+
  theme(axis.text.x=element_text(angle=45,hjust=1))
```

```
#####
```

```
#####
```

```
# Visualization 2 : Sales Volume using Color Gradient
```

```
#####
```

```
#####
```

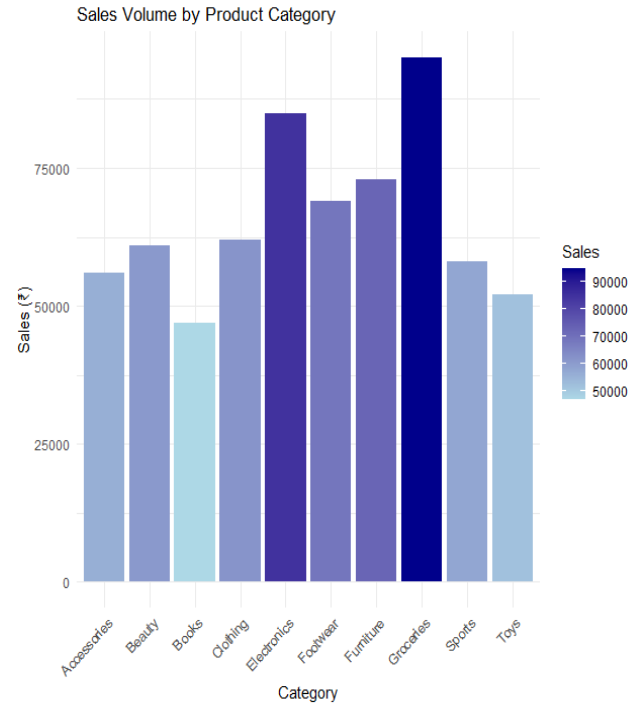
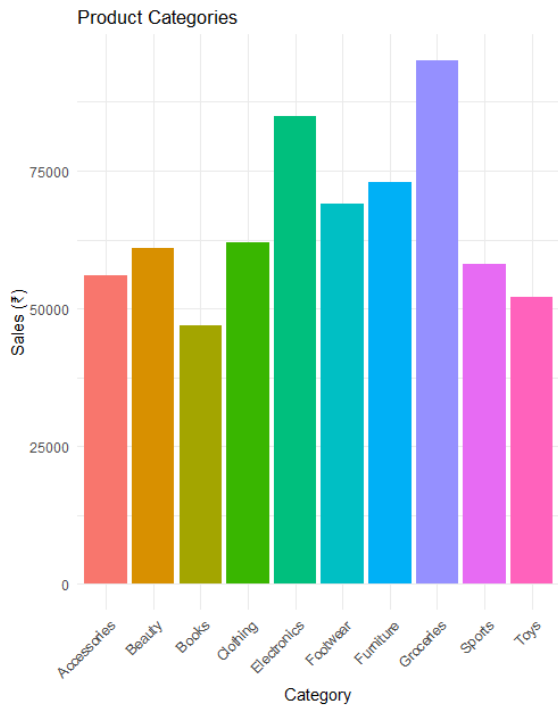
```
ggplot(sales,
  aes(x=Category,
      y=Sales,
      fill=Sales))+
  geom_bar(stat="identity")+
  scale_fill_gradient(
    low="lightblue",
    high="darkblue"
  )+
  labs(
    title="Sales Volume by Product Category",
    x="Category",
    y="Sales (₹)"
  )+
```

```
plt.xlabel("Category")
plt.ylabel("Sales (₹)")
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()
```

```
theme_minimal()+
theme(axis.text.x=element_text(angle=45,hjust=1))
```

OUTPUT:



Discussion

Visualization 1 (Qualitative Colors)

- Different colors distinguish product categories.
- Easy to identify each category.
- Does not indicate which category has higher sales through color alone.

Visualization 2 (Quantitative Colors)

- Light blue indicates lower sales.
- Dark blue indicates higher sales.
- Makes it easy to identify top-selling categories.

Challenges of Combining Qualitative and Quantitative Color Encoding

- Using many category colors together with a color gradient can confuse users.
- Too many colors reduce readability.
- Similar shades may be difficult to distinguish.

- A single graph with both category colors and value gradients can become cluttered.

Conclusion

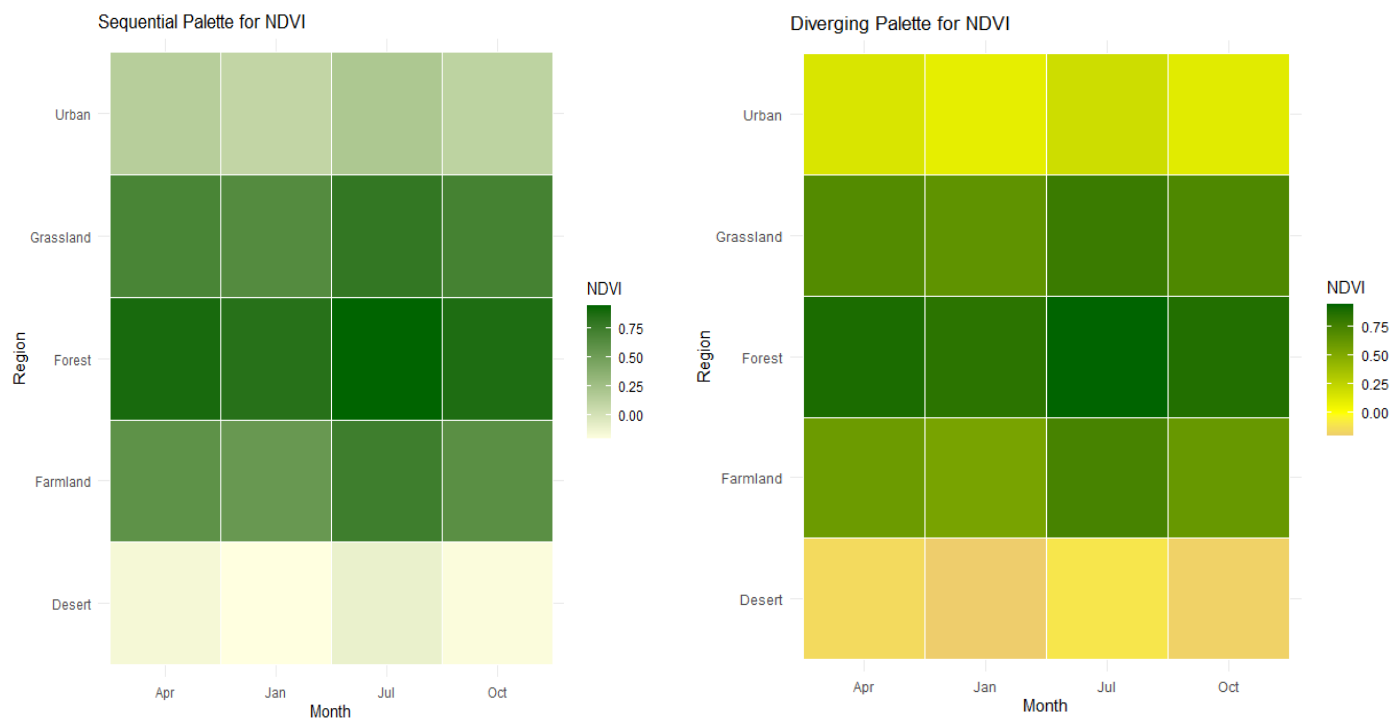
- Qualitative colors are best for distinguishing product categories.
- Sequential color gradients are best for representing sales volume.
- When both are needed, it is usually clearer to use two separate visualizations, as shown above, rather than combining both encodings into one chart.

7. Satellite Image Vegetation Analysis: A satellite dataset provides NDVI values ranging from -1 to $+1$. Environmental scientists must identify healthy vegetation, barren land, and water bodies.

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt import seaborn as sns data = { "Region":["Forest","Grassland","Farmland","Urban", "Desert"], "Jan":[0.85,0.65,0.55,0.10,-0.20], "Apr":[0.90,0.70,0.60,0.15,-0.15], "Jul":[0.95,0.80,0.75,0.20,-0.10], "Oct":[0.88,0.72,0.62,0.12,-0.18] } df = pd.DataFrame(data) # Convert to Long Format df_long = df.melt(id_vars="Region", var_name="Month", value_name="NDVI") # Convert to Matrix heatmap_data = df_long.pivot(index="Region", columns="Month", values="NDVI") plt.figure(figsize=(14,5)) # ===== ===== # 1. Sequential Color Palette # ===== ===== plt.subplot(1,2,1) sns.heatmap(heatmap_data, cmap="YlGn", annot=True, linewidths=0.5, cbar_kws={"label":"NDVI"})</pre>	<pre>install.packages("ggplot2") install.packages("tidyr") library(ggplot2) library(tidyr) ndvi <- data.frame(Region = c("Forest","Grassland","Farmland","Urban","Desert"), Jan = c(0.85,0.65,0.55,0.10,-0.20), Apr = c(0.90,0.70,0.60,0.15,-0.15), Jul = c(0.95,0.80,0.75,0.20,-0.10), Oct = c(0.88,0.72,0.62,0.12,-0.18)) ndvi_long <- pivot_longer(ndvi, cols = Jan:Oct, names_to = "Month", values_to = "NDVI") ##### ##### # 1. Sequential Color Palette ##### ##### ggplot(ndvi_long, aes(x = Month, y = Region, fill = NDVI)) + geom_tile(color = "white") + scale_fill_gradient(low = "lightyellow", high = "darkgreen") + labs(title = "Sequential Palette for NDVI", x = "Month", y = "Region", fill = "NDVI") + theme_minimal()</pre>

<pre>plt.title("Sequential Color Palette") # ===== ===== # 2. Diverging Color Palette # ===== ===== plt.subplot(1,2,2) sns.heatmap(heatmap_data, cmap="RdYlGn", annot=True, linewidths=0.5, center=0, cbar_kws={"label":"NDVI"}) plt.title("Diverging Color Palette") plt.tight_layout() plt.show()</pre>	<pre>##### ##### # 2. Diverging Color Palette ##### ##### ggplot(ndvi_long, aes(x = Month, y = Region, fill = NDVI)) + geom_tile(color = "white") + scale_fill_gradient2(low = "blue", mid = "yellow", high = "darkgreen", midpoint = 0) + labs(title = "Diverging Palette for NDVI", x = "Month", y = "Region", fill = "NDVI") + theme_minimal()</pre>
--	--

OUTPUT:



Evaluation

Sequential Palette (Light Yellow → Dark Green)

- Light colors represent **low vegetation**.
- Dark green represents **healthy vegetation**.
- Best for displaying vegetation density.

Diverging Palette (Blue → Yellow → Green)

- **Blue** = Water bodies (negative NDVI)
- **Yellow** = Barren land (around 0)
- **Green** = Healthy vegetation (positive NDVI)
- Clearly separates the three land-cover types.

Conclusion

The **Diverging Color Palette (Blue → Yellow → Green)** is the **best choice** because:

- It naturally distinguishes **water**, **barren land**, and **healthy vegetation**.
- It reduces interpretation errors.
- It is easier for environmental scientists to identify land-cover classes at a glance.
- It provides better readability than a simple sequential palette when both negative and positive NDVI values are present.

8. Cryptocurrency Market Volatility: Daily prices of Bitcoin, Ethereum, Solana, and Cardano span several years. Price values differ by several orders of magnitude, making direct comparison difficult.

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt data = { "Day": [1,2,3,4,5,6,7,8,9,10], "Bitcoin": [28000,30000,32000,34000,36000,38000,40000,42000,44000,46000], "Ethereum": [1800,1900,2100,2200,2400,2600,2800,3000,3200,3400], "Solana": [20,25,30,35,40,45,50,55,60,65], "Cardano": [0.30,0.35,0.40,0.45,0.50,0.55,0.60,0.70,0.80,0.90] } df = pd.DataFrame(data) plt.figure(figsize=(14,6)) # ===== # 1. Cartesian Plot # ===== plt.subplot(1,2,1)</pre>	<pre># Install ggplot2 (Run only once) install.packages("ggplot2") # Load Library library(ggplot2) # Cryptocurrency Dataset crypto <- data.frame(Day = 1:10, Bitcoin = c(28000,30000,32000,34000,36000,38000,40000,42000,44000,46000), Ethereum = c(1800,1900,2100,2200,2400,2600,2800,3000,3200,3400), Solana = c(20,25,30,35,40,45,50,55,60,65), Cardano = c(0.30,0.35,0.40,0.45,0.50,0.55,0.60,0.70,0.80,0.90)) ##### ##### # Convert Dataset to Long Format</pre>

```

plt.plot(df["Day"], df["Bitcoin"], marker='o',
label="Bitcoin")
plt.plot(df["Day"], df["Ethereum"], marker='o',
label="Ethereum")
plt.plot(df["Day"], df["Solana"], marker='o',
label="Solana")
plt.plot(df["Day"], df["Cardano"], marker='o',
label="Cardano")

plt.title("Cryptocurrency Prices (Cartesian Plot)")
plt.xlabel("Day")
plt.ylabel("Price (USD)")
plt.legend()
plt.grid(alpha=0.3)

```

```

# =====
# 2. Logarithmic Scale Plot
# =====

```

```

plt.subplot(1,2,2)

plt.plot(df["Day"], df["Bitcoin"], marker='o',
label="Bitcoin")
plt.plot(df["Day"], df["Ethereum"], marker='o',
label="Ethereum")
plt.plot(df["Day"], df["Solana"], marker='o',
label="Solana")
plt.plot(df["Day"], df["Cardano"], marker='o',
label="Cardano")

plt.yscale("log")

plt.title("Cryptocurrency Prices (Logarithmic Scale)")
plt.xlabel("Day")
plt.ylabel("Price (Log Scale)")
plt.legend()
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```

```

#####
#####

```

```

crypto_long <- data.frame(
  Day = rep(crypto$Day,4),
  Price = c(crypto$Bitcoin,
            crypto$Ethereum,
            crypto$Solana,
            crypto$Cardano),
  Currency = factor(rep(c("Bitcoin",
                          "Ethereum",
                          "Solana",
                          "Cardano"),
                        each=10))
)

```

```

#####
# 1. Cartesian Plot
#####

```

```

ggplot(crypto_long,
  aes(x=Day,
      y=Price,
      color=Currency,
      group=Currency))+

```

```

  geom_line(linewidth=1.2)+
  geom_point(size=3)+

```

```

  labs(
    title="Cryptocurrency Prices (Cartesian Plot)",
    x="Day",
    y="Price (USD)"
  )+

```

```

  theme_minimal()

```

```

#####
#####

```

```

# 2. Logarithmic Scale Plot

```

```

#####
#####

```

```

ggplot(crypto_long,
  aes(x=Day,
      y=Price,
      color=Currency,
      group=Currency))+

```

```

  geom_line(linewidth=1.2)+
  geom_point(size=3)+

```

```

  scale_y_log10()+

```

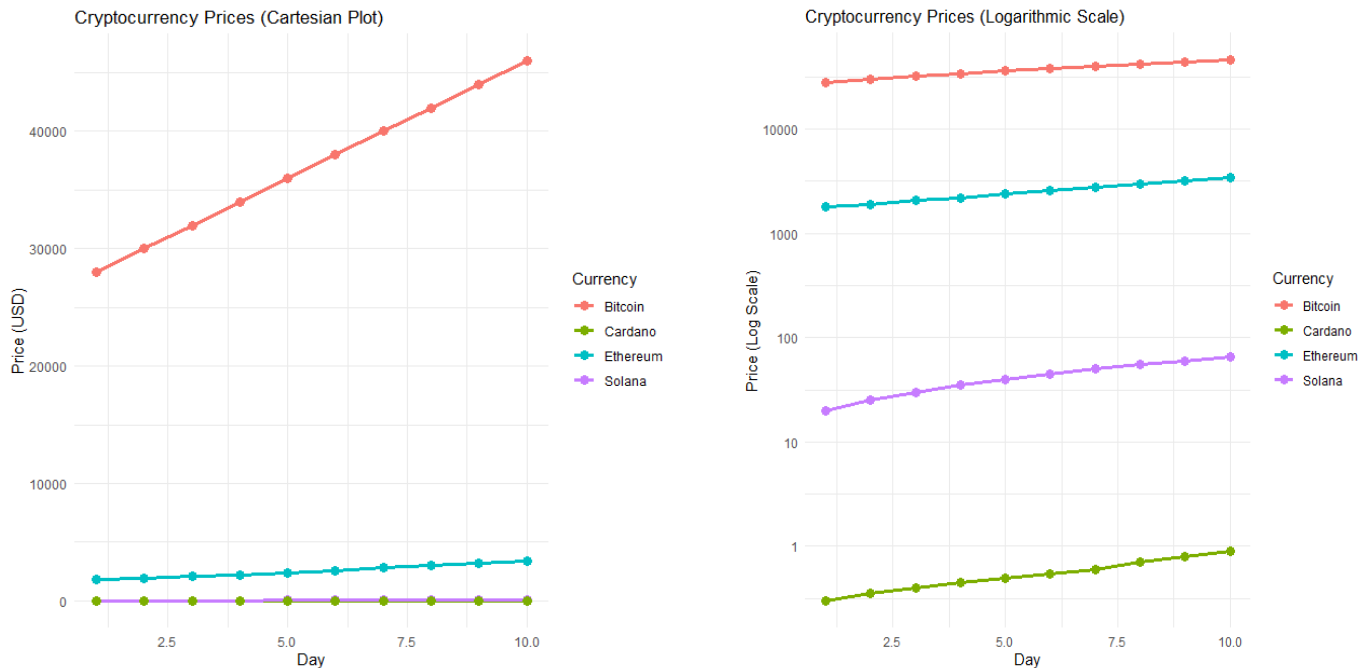
```

  labs(
    title="Cryptocurrency Prices (Logarithmic
Scale)",
    x="Day",

```

```
y="Price (Log Scale)"
)+
theme_minimal()
```

OUTPUT:



Benefits of Logarithmic (Nonlinear) Axes

- Makes cryptocurrencies with very different prices easier to compare.
- Small-value coins (Cardano, Solana) become clearly visible.
- Emphasizes percentage changes rather than absolute price differences.
- Useful when values span several orders of magnitude.

Limitations of Logarithmic Axes

- Less intuitive for beginners.
- Actual price differences are not immediately obvious.
- Zero or negative values cannot be displayed on a logarithmic scale.

Conclusion

The **Cartesian plot** clearly shows actual prices but hides the trends of lower-priced cryptocurrencies because Bitcoin dominates the scale.

The **logarithmic plot** is more effective for comparing the growth patterns of Bitcoin, Ethereum, Solana, and Cardano while maintaining visual clarity using different colors.

9. Marathon Runner Performance Analysis: A marathon organizer records runner speed, heart rate, distance covered, elevation, and running direction throughout the race. Coaches want to analyze pacing strategies and directional movement.

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt import numpy as np data = { "Distance":[5,10,15,20,25,30,35,40], "Speed":[12,13,14,13,12,11,10,9], "HeartRate":[120,130,140,150,155,160,165,170], "Elevation":[20,35,50,65,55,40,25,15], "Direction":[0,45,90,135,180,225,270,315] } df = pd.DataFrame(data) plt.figure(figsize=(15,6)) # ===== # 1. Cartesian Plot # ===== plt.subplot(1,2,1) scatter = plt.scatter(df["Distance"], df["Speed"], c=df["Speed"], cmap="viridis", s=180, edgecolors="black") plt.plot(df["Distance"], df["Speed"], color="gray") for i in range(len(df)): plt.text(df["Distance"][i], df["Speed"][i]+0.2, str(df["Distance"][i])+" km", fontsize=8) plt.colorbar(scatter,label="Speed (km/h)")</pre>	<pre># Install ggplot2 (Run only once) install.packages("ggplot2") # Load Library library(ggplot2) # Marathon Dataset runner <- data.frame(Distance = c(5,10,15,20,25,30,35,40), Speed = c(12,13,14,13,12,11,10,9), HeartRate = c(120,130,140,150,155,160,165,170), Elevation = c(20,35,50,65,55,40,25,15), Direction = c(0,45,90,135,180,225,270,315)) ##### ##### # 1. Cartesian Plot ##### ##### ggplot(runner, aes(x = Distance, y = Speed, color = Speed)) + geom_line(linewidth = 1.2) + geom_point(size = 5) + scale_color_gradient(low = "lightblue", high = "darkblue") + labs(title = "Marathon Performance (Cartesian Plot)", x = "Distance (km)", y = "Speed (km/h)", color = "Speed") + theme_minimal() ##### ##### # 2. Polar Coordinate Plot</pre>

```
plt.title("Marathon Performance (Cartesian
Plot)")
plt.xlabel("Distance (km)")
plt.ylabel("Speed (km/h)")
plt.grid(alpha=0.3)

# =====
# 2. Polar Coordinate Plot
# =====
ax = plt.subplot(1,2,2, projection="polar")

theta = np.deg2rad(df["Direction"])

bars = ax.bar(
    theta,
    df["Speed"],
    width=np.deg2rad(35),

color=plt.cm.viridis(df["Speed"]/df["Speed"]
.max()),
    edgcolor="black"
)

ax.set_title("Runner Direction (Polar Plot)")
ax.set_theta_zero_location("N")
ax.set_theta_direction(-1)

plt.tight_layout()
plt.show()
```

```
#####
#####

ggplot(runner,
    aes(x = factor(Direction),
        y = Speed,
        fill = Speed)) +

    geom_bar(stat = "identity",
        width = 1) +

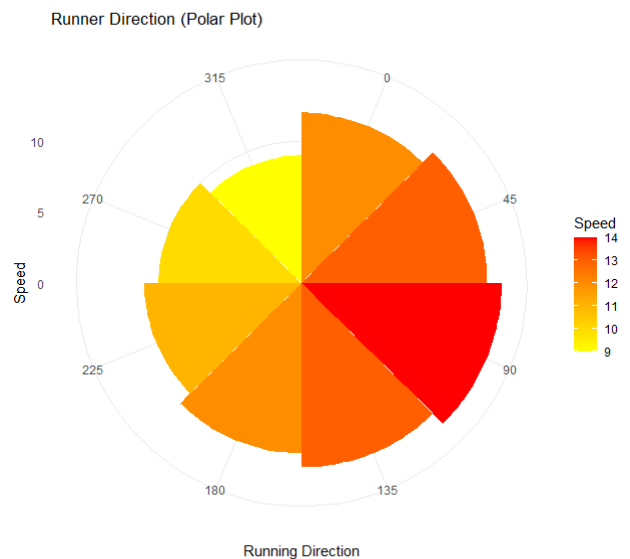
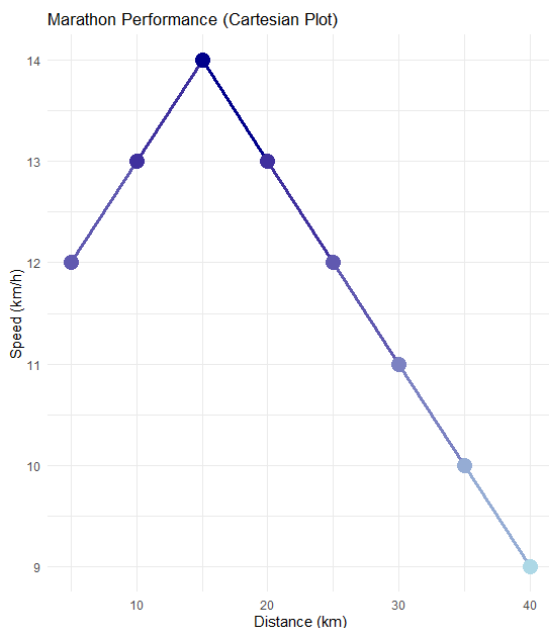
    coord_polar() +

    scale_fill_gradient(
        low = "yellow",
        high = "red"
    ) +

    labs(
        title = "Runner Direction (Polar Plot)",
        x = "Running Direction",
        y = "Speed",
        fill = "Speed"
    ) +

    theme_minimal()
```

OUTPUT:



Interpretation

Cartesian Plot

- Shows pacing strategy throughout the marathon.
- Speed changes are easy to compare across distances.
- Useful for identifying fatigue or acceleration.

Polar Coordinate Plot

- Displays running direction in a circular format.
- Color gradient highlights speed variations.
- Useful for analyzing movement direction and turns.

Recommendation

The **Cartesian Plot** provides deeper performance insights because it clearly illustrates how a runner's speed changes with distance, making pacing strategies easy to evaluate.

The **Polar Coordinate Plot** is better suited for analyzing directional movement, but it is less effective for studying changes in speed over the course of the race.

10. COVID-19 Vaccination Campaign Dashboard: Health officials monitor vaccination rates across districts, age groups, and vaccine types. They need visualizations suitable for both policymakers and the general public.

PYTHON	R PROGRAMMING
<pre>import pandas as pd import matplotlib.pyplot as plt import seaborn as sns data = { "District":["Chennai","Coimbatore","Madurai","Salem","Trichy"], "Vaccination":[92,78,65,85,95], "AgeGroup":["18-30","31-45","46-60","18-30","60+"], "Vaccine":["Covishield","Covaxin","Covishield","Covaxin","Covishield"] } df = pd.DataFrame(data) plt.figure(figsize=(14,6)) # ===== # 1. Policymakers Dashboard # ===== plt.subplot(1,2,1) bars = plt.bar(df["District"], df["Vaccination"],</pre>	<pre>install.packages("ggplot2") library(ggplot2) vaccination <- data.frame(District = c("Chennai","Coimbatore","Madurai","Salem","Trichy"), Vaccination = c(92,78,65,85,95), AgeGroup = c("18-30","31-45","46-60","18-30","60+"), Vaccine = c("Covishield","Covaxin","Covishield","Covaxin","Covishield")) ##### ##### # Visualization 1 : Policymakers Dashboard ##### ##### ggplot(vaccination, aes(x=District, y=Vaccination, fill=Vaccination))+ geom_bar(stat="identity")+ geom_text(aes(label=Vaccination),</pre>

```

    color=plt.cm.Blues(df["Vaccination"]/100)
)

for i in range(len(df)):
    plt.text(
        i,
        df["Vaccination"][i]+1,
        str(df["Vaccination"][i])+"%",
        ha="center",
        fontsize=9
    )

```

```

plt.title("COVID-19 Vaccination Dashboard
(Policymakers)")
plt.xlabel("District")
plt.ylabel("Vaccination Rate (%)")
plt.grid(axis="y", alpha=0.3)

```

```
#
```

```
=====
```

```
=====
```

```
# 2. Public Dashboard
```

```
#
```

```
=====
```

```
=====
```

```
plt.subplot(1,2,2)
```

```
status = []
```

```

for value in df["Vaccination"]:
    if value >= 90:
        status.append("High")
    elif value >= 70:
        status.append("Medium")
    else:
        status.append("Low")

```

```
df["Status"] = status
```

```
colors = []
```

```

for s in df["Status"]:
    if s == "High":
        colors.append("green")
    elif s == "Medium":
        colors.append("gold")
    else:
        colors.append("red")

```

```

plt.bar(
    df["District"],
    df["Vaccination"],
    color=colors,
    edgecolor="black"
)

```

```

vjust=-0.5,
size=4)+

```

```

scale_fill_gradient(
    low="lightblue",
    high="darkblue"
)+

```

```

labs(
    title="COVID-19 Vaccination Dashboard
(Policymakers)",
    x="District",
    y="Vaccination Rate (%)")
)+

```

```
theme_minimal()
```

```

#####
#####
# Visualization 2 : Public Dashboard
#####
#####

```

```

vaccination$Status <- ifelse(
    vaccination$Vaccination >= 90,
    "High",
    ifelse(
        vaccination$Vaccination >= 70,
        "Medium",
        "Low"
    )
)

```

```

ggplot(vaccination,
    aes(x=District,
        y=Vaccination,
        fill=Status))+

```

```
geom_bar(stat="identity")+
```

```

geom_text(aes(label=Vaccination),
    vjust=-0.5,
    size=4)+

```

```

scale_fill_manual(
    values=c(
        "High"="darkgreen",
        "Medium"="gold",
        "Low"="red"
    )
)+

```

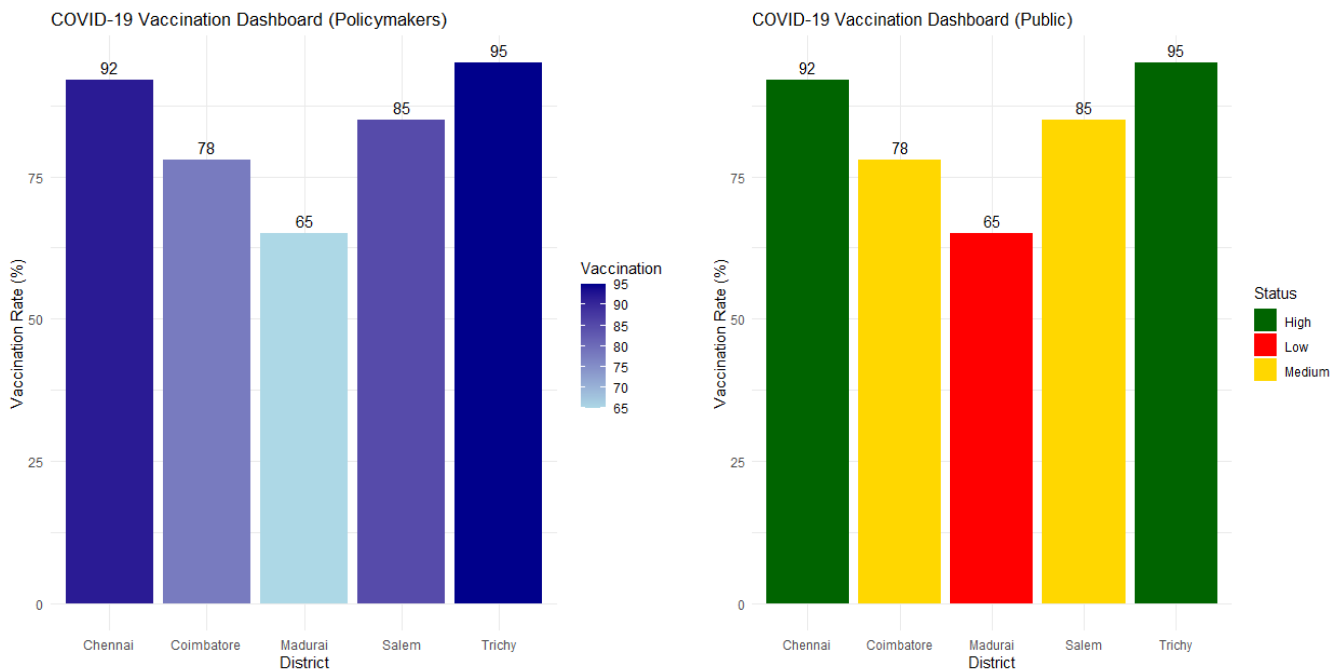
```

labs(
    title="COVID-19 Vaccination Dashboard
(Public)",
    x="District",

```

<pre> for i in range(len(df)): plt.text(i, df["Vaccination"][i]+1, df["Status"][i], ha="center", fontsize=9) plt.title("COVID-19 Vaccination Dashboard (Public)") plt.xlabel("District") plt.ylabel("Vaccination Rate (%)") plt.grid(axis="y", alpha=0.3) plt.tight_layout() plt.show() </pre>	<pre> y="Vaccination Rate (%)")+ theme_minimal() </pre>
--	---

OUTPUT:



Evaluation

Visualization 1 (Policymakers)

- Uses a blue gradient to represent vaccination percentages.
- Suitable for analysing exact vaccination rates.
- Helps identify districts with relatively higher or lower coverage.

Visualization 2 (General Public)

- Uses traffic-light colours:
 - High Coverage

- Medium Coverage
 - Low Coverage
- Easier for non-technical audiences to interpret quickly.

Accessibility

- Avoid relying only on red and green, since some users have red–green colour vision deficiencies.
- Display numeric labels on each bar to ensure the information is understandable even if colours are difficult to distinguish.
- Use high-contrast colours and a clear legend to improve readability.

Conclusion

- The blue gradient dashboard is more effective for policymakers because it emphasizes continuous vaccination rates.
- The categorized dashboard is more suitable for the general public because it provides a simple, easy-to-understand classification while remaining accessible through value labels and clear color contrast.